

RicozChange — Phase 2 PRD

Status: draft for review · **Timeline:** 4 weeks · **Team:** 1 dev + AI agents **Baseline:** Phase 1 MVP (live) — explainable risk scoring, collision detector, CAB, Slack-style outbox, post-change checks, CFR dashboard.

1. Why phase 2 exists

Phase 1 proves the product thinking. Phase 2 removes the three things that stop a real team from using it daily:

1. **Approvals live in a demo outbox**, not in Slack where approvers actually are.
2. **Everyone is the same demo user** — no login, no accountability, anyone can approve.
3. **Changes must be typed into the web form**, so frontline engineers won't adopt it.

Fixing these three converts a demo into a tool a team runs its week on.

2. Goals and non-goals

Goals - An approver can install the Slack app, and approve/reject real changes from Slack DMs — with the risk "why" inline. - Real login (Google or email via Clerk); every action attributed to a real user; role-based permissions enforced server-side. - An engineer can create a change by sending an email; the system parses it, scores it, replies with the score, and files the draft. - Approval reminders and post-change check prompts go out by email automatically. - Zero regressions: all Phase 1 flows and the public demo keep working.

Non-goals (explicitly out of scope for 4 weeks) - Multi-workspace Slack (single workspace is enough to launch) - Microsoft Teams parity - Calendar integration (Google/Outlook freeze sync) - Mobile push notifications - Learning risk scores (Tier 2 — needs outcome data volume we don't have yet)

3. Feature 1 — Real Slack integration (week 2)

What exists already

The outbox message format (header, RISK/CLASS/WINDOW/SYSTEMS fields, "WHY THIS SCORE" factors, Approve/Reject actions) is implemented in `notifications.py` and rendered by `/slack`. The decision path (`record_approval`, any-of policy) is a single service call. Phase 2 keeps both and swaps the transport.

User stories

- As an approver, after the RicozChange app is added to our workspace, I receive a DM when a change needs my decision, containing score, window, systems and the "why".
- As an approver, clicking  Approve in Slack resolves the change (any-of policy) and updates the message ("Approved by @you · 10:32").

- As an admin, I can install/reinstall the app from the RicozChange UI and see connection status.
- As an engineer, when my change is approved/rejected, I get a Slack update with who decided and any comment.

Technical design

- **App config:** Slack app manifest (bot scope `chat:write`, `im:read`, `im:history`, `users:read`; interactive components enabled). Signing secret + bot token stored server-side (env first; DB `settings` row so reinstall doesn't need a redeploy).
- **Install flow:** `GET /api/integrations/slack/install` → OAuth 2 redirect to Slack → `GET /api/integrations/slack/oauth/callback` exchanges the code, stores the bot token, maps installing admin. CLI/manifest automated so it's one click.
- **Outbound DMs:** when `queue_approval_requests` runs (already exists), additionally call Slack `chat.postMessage` for each pending approver with `slack_id` set; keep the DB outbox row as the source of truth + delivery audit (`sent_at`, `slack_ts`, `error`).
- **Inbound actions:** `POST /api/slack/interactions` verifies `X-Slack-Signature` (HMAC with signing secret, ±5 min window), parses the payload, resolves action → (notification, approval), calls the existing `record_approval` with `source="slack"`, then `chat.update` on the original message. Idempotent: acting on an already-acted notification returns a friendly ephemeral message.
- **User mapping:** new `slack_id` column on `users`; admin UI (or auto-link on first Slack action by email match) maps workspace members to app users. Unmapped approvers get an ephemeral "link your account" message.

Data model deltas

- `users.slack_id TEXT UNIQUE NULL`
- `settings(key TEXT PK, value JSON)` — Slack bot token, signing secret version
- `notifications.sent_at, slack_channel, slack_ts TEXT NULL` (delivery audit)

Acceptance criteria

1. Fresh workspace install → install a change is submitted → DMs arrive to mapped approvers within 5 s with the full "why".
2. Clicking Approve in Slack flips the change to `approved`; the other pending approver's message auto-closes (any-of) and their message updates to "superseded".
3. Replay/duplicate interaction clicks never double-apply (idempotent).
4. Invalid Slack signature → 401, logged.
5. Slack outage → outbox marks delivery failed, web approvals still work.

4. Feature 2 — Auth and roles (week 1 — first, because both other features depend on real identity)

What exists already

`auth.py` has a Clerk JWKS verification path behind `GATE_BY_DEMO_USER`; `users` has a `clerk_id` column and 4 seeded roles (admin, manager, approver, engineer). The seed's `APPROVER_ROLES` already drives who

gets approval requests. Phase 2 turns this from scaffolding into enforcement.

User stories

- As a user, I log in with Google (Clerk hosted UI); my role is assigned by an admin.
- As an engineer, I can create/submit my own changes and record post-change results for them, but I cannot approve anything.
- As an approver, I can approve/reject pending approvals and vote in CAB, but cannot manage freezes or users.
- As an admin, I manage users/roles, standard-change templates, freezes and CAB.
- As anyone, I see the dashboard read-only.

Permission matrix (enforced in FastAPI dependencies, not the frontend)

Action	engineer	approver	manager	admin
View dashboards/changes/audit	✓	✓	✓	✓
Create / edit / submit own change	✓	✓	✓	✓
Approve / reject (assigned)	✗	✓	✓	✓
CAB vote	✗	✓	✓	✓
Create CAB, add items, decide	✗	✗	✓	✓
Manage freezes	✗	✗	✓	✓
CSV import	✗	✗	✓	✓
Users, roles, templates, settings	✗	✗	✗	✓

Rules that don't fit the matrix: an engineer can only edit/submit **their own** draft changes; post-change check is recorded by the change **owner**; you can never approve a change you own.

Technical design

- Frontend: Clerk React provider replaces the demo-actor bootstrap; `VITE_CLERK_PUBLISHABLE_KEY`.
- Backend: Clerk JWT verification (already implemented in `auth.py`) becomes the default; demo mode remains available behind `GATE_BY_DEMO_USER` for local dev and tests.
- New dependency `require_role(*roles)`; applied per-route per the matrix; JWT `sub` maps to `users.clerk_id` (provision user on first login with role `engineer`; admin promotes).
- Alembic introduced for migrations (Postgres on Render makes `create_all` -only a growing risk); baseline migration = current schema.

Acceptance criteria

1. Logged-in engineer calling an approve endpoint gets 403; audit log records the attempt.
 2. Owner cannot approve own change (403).
 3. Demo mode (no Clerk keys) still passes the entire existing test suite unchanged.
 4. All existing endpoints return 401 without a valid session.
-

5. Feature 3 — Email-to-change + email notifications (week 3)

User stories

- As an engineer in the field, I email `change@ourdomain` with subject = title and a body (description / systems / window in simple keyword lines), and get a reply with the filed draft, its risk score and the "why".
- As an approver, I get a daily 9:00 digest of pending approvals (and an immediate email if Slack isn't set up for me).
- As an owner, after my change's window ends I get a "did it work?" email with a success/partial/failed mailto link; clicking records the post-change result.

Technical design

- **Inbound provider:** SendGrid Inbound Parse (free, webhook POST to `POST /api/integrations/email/inbound`, signature verification). IMAP polling kept as a fallback module for self-hosted mailboxes — same parser, different transport.
- **Parser (deterministic, testable):** subject → title; body keywords `systems: payments-db, orders-api / window: 2026-09-30 22:00-00:00 / type: normal|standard|major|emergency / rollback:` free text. Unparseable → filed as `draft` with description untouched + error noted in reply. Reply always includes score, "why" line items, and a web link to the draft.
- **Safety:** dedupe by `Message-ID`; allowlist of sending domains (configurable); auto-detected spam/attachment emails rejected; inbound email cannot fast-track or approve — files a draft only, humans submit.
- **Outbound:** one sweep per minute (FastAPI lifespan task, no Redis/Celery): post-change prompts after `window_end` for `completed` / `failed` changes with no result; daily pending-approval digest; Slack-failure fallback emails.

Data model deltas

- `email_inbound(id, message_id UNIQUE, from_addr, subject, body, parsed_change_id, status[created|rejected|error], error_detail, received_at)`

Acceptance criteria

1. A conforming email files a draft with correct systems/window and replies with score + "why" within 60 s; the same email twice files one change (dedupe).
2. Garbage email → rejected with a helpful reply, nothing filed.
3. Digest email lists only the recipient's pending approvals.
4. Post-change prompt email → mailto click → result recorded with `source="email"`.

6. Week-by-week plan

Week	Deliverable	Demo gate
1	Clerk login, role enforcement, Alembic baseline, per-user audit	Two browsers, two roles: engineer can't approve; owner can't approve own change

Week	Deliverable	Demo gate
2	Slack app install, real DMs with "why", interactive approve/reject, user mapping	Approve from phone Slack in < 30 s; duplicates idempotent
3	SendGrid inbound → parsed draft + scored reply; digests; post-change prompts	File a change entirely from a phone email client
4	Hardening (rate limits on webhooks, secret rotation, log hygiene), E2E tests, updated demo script + docs, deploy	Full manager demo: Slack approve + email-filed change on the public URL

Estimated effort: W1 ~6 dev-days, W2 ~7, W3 ~7, W4 ~5 (solo + AI agents, includes tests).

7. Risks

Risk	Mitigation
Slack DMs to users without linked accounts	Ephemeral link prompt + email fallback
Clerk free-tier limits (5k MAU) — fine for pilots	Document upgrade path; JWT verification is provider-agnostic
Email parsing variance	Deterministic keyword parser + "unknown → draft" fallback, never silent failure
Webhook abuse on public URL	Signature verification + allowlists + rate limit
Public demo DB is shared	Phase 2 demo uses a fresh Render environment, not the manager-demo URL

8. Success metrics (30 days after rollout)

- $\geq 80\%$ of approvals decided from Slack (not web)
- Median approval latency < 1 hour (vs. next-business-day baseline)
- $\geq 25\%$ of changes created via email by week 4
- 100% of actions attributable to a real logged-in user
- Zero unauthorized-action incidents

9. Deliberately deferred

Teams integration, calendar sync, mobile push, multi-workspace Slack, learned risk weights, Jira/ServiceNow — each has a note in the roadmap and none blocks a paying pilot.